Shipment tracking for Mars Industries: proposal and prototype

Live prototype: https://mb-shipment-tracking.vercel.app/ · Repository: https://github.com/Markuson/MB_Shipment_tracking · Marc Usón · 27 October 2026

1. Product in one paragraph

Mars Industries ships from several sites, by road and sea, through operators that each report in their own format. The product gives customers one place to see where a shipment is, whether it is on time and when it will arrive, and gives operations an inbox of only what needs attention, ranked, each with a suggested next action. One principle drives every screen: never invent a status or an ETA. Confirmed facts, estimates and unknowns look different (solid, dashed and muted lines), and what is unknown is shown as unknown.

2. Where AI goes, and why

AI is used in three places. In each one the basis is visible, and a person controls anything that reaches a customer.

2.1 Status normalisation

- **Problem:** operators send `ANC`, `LOD` or "PARADA TÉCNICA EN FRONTERA". Nobody should have to read three vocabularies.
- **Prototype:** a rule table maps known wording deterministically. A simulated model handles only wording no rule covers and returns a High, Medium or Low band, never a percentage. Low means "needs review": ops see an exception, and the customer sees "Awaiting details from operator", not the guess. Ops see the operator's original wording and the method (rule or AI) beside every status. The customer never does.
- **Production:** rules first; an LLM only for unseen wording, constrained to a closed vocabulary. Bands are calibrated against historical outcomes, not the raw model score. Ops corrections, keyed by operator and exact wording, are reviewed before they become rules. (The data structure exists in the prototype; the correction screen was left out.)
- **Human control:** an uncertain interpretation never reaches the customer.

2.2 Delay risk and ETA

- **Problem:** the customer's question is "when does it arrive?", and carriers rarely give an honest date.
- **Prototype:** the ETA is always a range, labelled as an estimate, with a cited reason and a visible "simulated route history" label. It stays unknown when customs hold the shipment, a status is under review, an operator is silent, or there is no history. The range is compared with the promised date to give on time, possible delay or delayed. In the demo, reporting a berth wait at the port moves the hero shipment from "On time" to "Possible delay, 16–18 Nov".
- **Production:** a statistical model per route and operator produces the numbers; the LLM only explains them in words and never edits them.
- **Human control:** informational only. It never acts, and it is replaced by the confirmed delivery.

2.3 Next action and drafted customer notice

- **Problem:** ops lose time deciding what to do with each exception and writing to the customer, who finds out late or by phone.
- **Prototype:** each exception shows a suggested action from a fixed cause-to-action table, labelled "Suggested (simulated)". "Notify customer" opens a drafted notice with a "Based on:" line citing the events it relies on. Once ops approve it, it appears in the customer view as its own block, "Message from your logistics team", under the automatic status alert. The other actions only log a request. An exception with an action taken sinks to the bottom of the inbox but stays visible: acting is not resolving, and it clears only when the situation changes.
- **Production:** a model ranks actions by how similar past exceptions were resolved; an LLM drafts the notice from the cited events only.
- **Human control:** nothing is sent without approval, because a message to a customer cannot be unsent.

2.4 What AI does not do

It never confirms a fact, never changes an ETA and never acts without approval. Two things were cut on purpose: a plain-language summary (the timeline, the ETA and the alert already say it) and free-form customer Q&A (mock logic answering open questions would undermine the reliability the product is built on). In production, Q&A would answer only from cited events and say "I don't know" when there is no basis.

3. System approach

3.1 Architecture

Operator → adapter → raw events stored verbatim → normalisation (rules, then AI) → derived state (statuses, exceptions, ETA) → alerts and views. The prototype has the same shape; only ingestion is mocked, and the demo panel plays the part of an operator sending events. State is never stored: the store holds only inputs (the clock and the events), and pure functions derive everything shown. One event therefore updates the inbox, the detail, the ETA and the customer alert consistently.

3.2 Key technology choices

Choice	Why	Alternative
Next.js App Router, TypeScript	The company's default. The real product needs a server for the data perimeter, authentication and operator webhooks, and the prototype keeps that boundary: server pages call a data-access module that takes the viewer as a parameter.	A Vite SPA: simpler for a mock-only demo, but it would be thrown away.
`Reliability<T>` (confirmed, estimated, unknown)	The compiler forces every datum to say what kind of knowledge it is, so an estimate cannot be drawn as a fact.	Convention and review.
Pure domain functions, zustand for inputs only	One source of truth, easy to test, no contradictions between screens.	Storing derived state.
Tailwind CSS v4 with semantic tokens	Consistent look; no palette classes in components.	A full component library.
Vitest and React Testing Library	Tests where the correctness risk is: the pure logic.	End-to-end tests (Playwright), not included.
Native `<dialog>` for the notice	One overlay does not justify a dependency.	Radix Dialog.

3.3 Planned integrations

Ingestion differs per operator: webhooks, API polling, EDI messages, email or PDF parsing, one adapter each. Third-party visibility aggregators exist, so build versus buy is a decision to take with the client. Authentication and the data perimeter move to the server, which returns customer-safe views. Notifications go out by email and push, and a database holds events, corrections and approved notices. Which feeds each operator offers is an assumption to validate.

3.4 Freshness and notifications

The view would refresh by polling, or by server-sent events where volume justifies it, through TanStack Query. A leg stays quiet until it passes a stale threshold for its type (road 8 h, port 24 h, sea 48 h, customs 24 h, last mile 6 h), and silence only counts on the leg whose turn it is to report, so a customs hold is not also reported as a silent last mile. Customers are alerted from the same derived state; messages written by hand need ops approval.

3.5 Reliability model

Events are facts, stored as received. Interpretations carry their provenance (rule, AI band, correction). Dates are of three kinds: planned (the booking), confirmed (an operator event) and estimated (our model, always a range).

4. Assumptions, limitations and next steps

4.1 Core assumptions (each to validate with the client)

- **Booking dates.** Every leg is booked with a planned start and end. That date separates "not due yet" from "due and silent". If a carrier gives no dates, a leg would be due once the previous one finishes, and we would never fill in a default date, because an invented date is an invented fact.
- **Thresholds and bands.** The stale thresholds per leg type and the confidence bands are modelled, not measured.
- **The promise** is the planned end of the last leg.

4.2 Known limitations

- The ETA rests on invented route history, and the AI is simulated.
- Tests cover the pure logic; there are no end-to-end tests.
- Six hand-written shipments, one ops user, site filter only. No real authentication, maps or document extraction.
- The demo resets on a full page reload.
- Derivation runs in the browser so the demo can recalculate live, so the customer's page data still includes the operator's wording, though it is never rendered. In production the API returns customer-safe views.
- A status under review shows the customer "Awaiting details" even if an earlier reliable status exists. It never shows a guess, but it can look like a step back.
- Silence is detected only on legs with no events.
- An exception with an action taken never moves back up on its own, however long the problem lasts.
- There is no authentication: the customer account comes from the URL or the demo panel, which stands in for the login.

4.3 Prioritised next steps

1. Operator ingestion adapters and the event store: nothing real works without them.
2. Server-side perimeter and authentication: a customer must never see another's data.
3. A calibrated ETA model per route and operator, replacing the mock table.
4. A review workflow so ops corrections become rules safely.
5. Notifications by email and push, and customer subscriptions.
6. Operator and mode filters, documents, and a customer Q&A that answers only from cited events.

5. How I used AI to build this

I built this with AI coding tools, as the brief expects. Claude Code wrote the code and tests one step at a time: it proposed a plan, I read and amended it, and only then did it implement, run type checks, lint, tests and a production build, and look at the page in a browser. A separate Claude chat project helped me challenge the scope, review each plan and result against the project rules and the reliability principle, and draft decision entries. I followed my own research, plan, implement flow with fresh context per phase, as in the handbook, and dropped BMAD as too heavy for this size.

What I changed or caught: after 4.5 hours I cut scope (six shipments, one ops filter, no correction screen). Reviews found real defects: a failed delivery raised no exception, an ambiguous status was invisible in the inbox, and a customs hold was also reported as a silent last mile, which I spotted in the browser. I also steered the design: components take data as props, the estimate is labelled as simulated, and the status alert and the approved notice stay separate. I removed a generated test that tested a library, and checked the others against the product rules. Details are in `docs/AI_USAGE.md`.